

류기현 Frontend Developer

React와 TypeScript로 의료 제품과 사내 시스템을 만들고 있습니다. 최근에는 프로젝트 15개가 의존하는 디자인 시스템의 컴포넌트 API와 배포 구조를 설계했고, npm에 공개한 오픈소스 라이브러리의 메인테이너로 릴리스와 소비자 환경 호환을 관리했습니다.

공통 코드의 인터페이스는 감으로 정하지 않고 데이터로 정하는 편입니다. 레포지토리 전체에서 어떤 옵션이 실제로 쓰이는지 집계하는 프로세스를 만들어 API 결정 근거로 쓰고 있습니다.

khryu0610@gmail.com [GitHub](#)[↗] [Blog](#)[↗]

경력

메디컬에이아이 서울 2024.11 - 현재
소프트웨어그룹 프로젝트본부 FE팀 · Frontend Developer

학력

연세대학교 서울
문과대학 중어중문학과
2018.03 - 2023.08

교육

SSAFY 삼성청년소프트웨어 아카데미 서울
Java 트랙
2023.07 - 2024.06
멋쟁이사자처럼 웹 프론트엔드 스쿨 온라인
프론트엔드 중점 과정
2022.07 - 2023.01

오픈소스

vue3-kakao-maps [GitHub](#)[↗] 6인
카카오맵 API를 Vue 3 컴포넌트로 사용할 수 있게 만든 오픈소스 라이브러리

적용 범위 npm 공개 패키지 · 월 다운로드 약 500회

[Vue](#) [TypeScript](#) [Vite](#)

핵심 기여

라이브러리의 지원 범위를 산출물 기준으로 정의하고, 기능 추가가 이를 깨지 않게 설계

-  Vue를 번들에서 제외하면서 peerDependencies에는 선언하지 않아, 사용자는 어떤 Vue 버전에서 동작하는지 알 수 없었습니다.
-  라이브러리가 어떤 Vue 버전에서 동작하는지를, 개발에 쓰는 Vue 버전이 아니라 빌드된 코드가 실제로 호출하는 Vue 함수를 기준으로 정했습니다(Vue 3.2 이상). 이후 `v-model` 을 추가할 때 `defineModel` 을 쓰면 빌드된 코드가 Vue 3.3부터 있는 함수를 호출하게 되는 것을 확인하고, 3.2 사용자가 깨지지 않도록 같은 기능을 `props` 와 `update` 이벤트로 직접 구현했습니다. 공개 API가 바뀌는 변경은 PR마다 명시하고 릴리스 버전 정책에 반영했습니다.
-  월 약 500회 다운로드되는 패키지에서, 기존 사용자가 Vue를 업그레이드하지 않고도 새 기능을 쓸 수 있게 했습니다.
-  대안: 지원 하한을 3.4로 올리기
기능 하나를 위해 기존 사용자에게 업그레이드를 강요하게 됩니다.

대표 프로젝트

MAIUI 3.0 메디컬에이아이

2026.03 ~ 현재

프로덕트 15개가 의존하는 디자인 시스템의 컴포넌트 API와 배포 구조를 다시 설계한 개편

적용 범위 프로덕트 15개 3.0 전환 컴포넌트 41개 중 11개 · 테스트 465건 자산 반영 대기 평균 90일 → 수일

React TypeScript Base UI styled-components Storybook Node.js

핵심 기여

기존 Flat 컴포넌트의 불편함 개선

- ❗ 기존 디자인 시스템은 컴포넌트마다 API가 달랐고, 제품은 CSS 선택자로 내부 DOM을 직접 선택해 커스텀하고 있었습니다. 내부 구조가 바뀌면 깨질 위험이 있고, 프로덕트단에서 AI가 일관된 코드를 작성하기 어려웠습니다.
- ✅ 스타일링을 위해 의미없이 존재하는 wrapper를 지양하고, 컴포넌트 표면에서 받은 prop이 단일 DOM 요소로 들어가게 했습니다. 여러 요소가 한 컴포넌트를 구성하는 경우에는 prop이 배정되는 요소를 선언적으로 관리하고 스토리북으로 공유했습니다.
예: textarea는 scroll area로 감싸는 대신 textarea 요소 자체에 적용할 수 있는 스크롤바 스타일 헬퍼를 개발했습니다.
↔ 반면 wrapper가 불가피했던 input은 컨테이너와 실제 input 요소에 배정되는 prop을 구분했습니다.
- ❌ 대안: Flat 컴포넌트에서 내부 각 요소마다 prop을 전달하는 패턴 도입
필요한 유틸을 개발하고 유지보수하는 비용만큼 효과가 있을지 의문이었습니다. 세부적인 커스텀이 필요하면 Compound 컴포넌트를 쓰도록 권장했습니다.
- ❌ 대안: 개발 편의성을 위해 wrapper를 자유롭게 사용
불필요한 요소를 줄이고 사용자의 직관대로 움직이는 것이 디자인 시스템의 품질을 높이는 지점이라고 판단했습니다.

- ❗ 작은 요구사항에도 디자인 시스템을 수정하고 다시 배포해야 했습니다. 제품 개발에 큰 병목으로 작용했습니다.
- ✅ API에서 열고 닫을 옵션은 제품의 실사용 현황을 전수 조사한 결과를 근거로 정하고, 내부 커스텀이 필요한 경우에는 Compound 컴포넌트를 조합해 사용하도록 했습니다.
예: 스크롤바 너비를 커스텀하는 용례가 없음을 확인해 해당 옵션을 닫을 수 있는 근거로 제시했습니다.

완성 전에 제품이 쓸 수 있는 점진적 배포 구조 설계

- ❗ 디자인 시스템 개편이 끝나기 전에 신규 제품 개발이 시작되어야 했습니다. 당시 25%의 컴포넌트만 개발이 완료됐었고, 나머지는 이전 버전 구현이 그대로 실려 있거나 코드가 없었습니다. 팀이 두 명이라 전체를 먼저 완성하는 선택은 제품 일정을 그만큼 미루는 것이었습니다.
- ✅ 컴포넌트마다 Experimental·Preview·Stable 상태를 두고 상태별로 배포 경로를 나눴습니다. 안정된 것은 기본 진입점, 검증이 남은 것은 preview 진입점, 제품이 소스를 고쳐 써야 하는 것은 소스를 복사해 가는 레지스트리로 배포했습니다.
- ✅ 제품이 디자인 시스템 완성을 기다리지 않고 개발을 시작했습니다. 제품 쪽에는 디자인 시스템 컴포넌트를 감싸는 UI 레이어를 두어서 변경 범위를 최소화했습니다.
- ❌ 대안: 제품에서 figma mcp로 빠르게 개발한 후, 디자인 시스템이 완성되면 마이그레이션
마이그레이션 비용이 크고, 컴포넌트를 Stable 상태로 옮기기 전 실사용 검증 단계로 활용하기 좋은 기회라고 판단했습니다.

디자인 자산 갱신 자동화

- ❗ 아이콘과 디자인 토큰의 변경 사항을 사람이 확인하고 코드로 옮기고 있었습니다. 변경 이력을 분석해보니 갱신 간격은 평균 90일, 임의 시점의 예상 대기 기간은 64일이었고, 개편 당시 프로덕트 전체에서 사용되던 아이콘 376개 중 245개만 디자인 시스템에 반영된 상태였습니다.
- ✅ 변경 감지부터 코드 생성, 변경 내역 작성, PR 생성까지 이어지는 파이프라인을 만들고 디자인 자산을 코드와 전체 동기화했습니다.
- ✅ 분기 단위로 발생하던 반영 대기를 며칠 수준으로 줄였습니다. 해당 시스템 구축에는 12작업일이 들었고, 반복 작업 절감까지 포함하면 1년 이내에 구축 비용을 회수할 수 있습니다.

여러 제품 저장소의 코드를 모아 변경 영향과 공통 API 실사용 현황을 조사한 도구

조사 대상 사내 저장소 15개 모달 사용처 132건 · 숨은 경로 27%

TypeScript Node.js TypeScript Compiler API LLM

핵심 기여

정적 검색으로 찾기 어려운 변경 영향을 AI로 분석

- 공통 모달 개편을 앞두고, 리뷰와 QA의 완성도를 높이고자 제품에서 영향을 받는 화면을 조사했습니다. 단순 사용처는 코드 검색으로 찾을 수 있었지만, 실제 진입 경로와 노출 조건, 제품별 수정 정도, 변경 위험도는 코드를 직접 읽어야 판단할 수 있었습니다.
- 사람이 리뷰할 때 확인하던 내용을 15개 항목으로 나누고, AI가 저장소의 코드 맥락을 읽어 같은 형식으로 결과를 작성하도록 했습니다. 사용 위치, 진입 경로, 노출 조건처럼 코드 맥락이 필요한 부분을 AI에게 위임했습니다.
- 3일 동안 4개 제품의 모달 사용 사례를 모았고, 그중 4분의 1이 넘는 사례가 서버 오류, 시간 초과, 유효성 검사 실패처럼 정상적인 화면 흐름만 따라가서는 놓치기 쉬운 사용처였습니다. 결과는 유형과 위험도, 확인 여부로 탐색하는 대시보드로 모아 코드 리뷰와 디자인 QA가 같은 범위를 검토하도록 했고, AI가 판단한 내용에는 근거와 확신도를 함께 남겼습니다.

헬퍼 함수의 옵션을 삭제해도 되는지 호출부 전수 조사로 확인

- 스크롤바의 커스텀을 어디까지 허용할지 결정하는 과정에서, 스크롤바의 너비를 고정해도 될지에 대한 의사결정 근거가 필요했습니다.
- 디자인팀과 합의도 중요하지만 지금까지의 데이터를 기반으로 앞으로의 커스텀 가능성을 예측하고자 했습니다. 15개 레포지토리에서 호출부 인자를 수집하는 건 정적 분석 스크립트에 맡겼고, 코드를 읽고 해석해야 하는 부분은 AI가 맡았습니다.
- 호출 41건을 분석한 결과 실제로 쓰인 사례가 없었습니다. 같은 수집 결과에서 사용처가 없는 `disabled` · `readOnly` 옵션과 트랙 여백이 음수로 계산되는 20곳도 함께 찾았습니다.
- 대안: 스크롤바뿐 아니라 다른 함수나 컴포넌트의 용례를 수집하는 범용 도구부터 만들기
지금 필요한 질문에 빠르게 답하고 넘어갔습니다. 레포 수집과 호출부 추출 단계만 다른 옵션을 조사할 때 쓸 수 있도록 설계했습니다.

- 결론을 낸 뒤 반례를 찾으려고 `scrollbarWidth` 라는 이름을 다시 검색하자 10, 15, 20이라는 값이 나왔습니다.
- 프로덕트에서 새롭게 정의한 `scrollbarWidth` 라는 prop으로 받은 값을 `areaWidth` 에 넘기고 있었습니다. 디자인 시스템의 옵션 이름이 동료의 멘탈모델과 일치하지 않다고 판단했습니다.
- `scrollbarWidth` 옵션은 제거하되, 해당 옵션과 헷갈리게 쓰인 다른 옵션의 이름을 변경함으로써 동료들이 잘 쓸 수 있게 지원했습니다.

사내 업무관리 시스템, 고객지원 시스템 메디컬에이아이

2025.07 ~ 2026.03

회사의 자산, 매출, 거래처를 관리하는 업무관리시스템과 고객 CS를 응대하는 고객지원시스템을 개발하고, 공통으로 사용되는 대용량 파일 다운로드 기능을 개선

메모리 사용량 약 1.9GB → 수십 MB

React TypeScript TanStack Query Zustand Zod React Hook Form

핵심 기여

대용량 파일 다운로드를 스트림 기반으로 변경

- ❗ 서버가 파일 전체를 압축한 뒤 응답하고 브라우저가 다시 후처리하는 구조에서 파일이 커질수록 요청이 실패했습니다. 1GB 파일을 처리할 때는 브라우저 메모리 사용량이 약 1.9GB까지 증가했습니다.
- ✅ 처음에는 네트워크 문제를 의심했지만 백엔드와 처리 과정을 확인하면서, 전체 파일이 준비될 때까지 기다리고 다시 메모리에 쌓는 구조 자체가 병목이라고 판단했습니다. 서버는 데이터를 생성되는 대로 보내고, 클라이언트도 받은 데이터를 메모리에 계속 쌓지 않고 가능한 환경에서는 바로 파일로 기록하도록 변경했습니다.
- 🔍 100MB부터 1GB까지 파일 크기와 동시 요청 수를 달리해 검증했으며, 기존 방식에서 실패하던 대용량 다운로드를 모두 완료할 수 있었습니다. 1GB 처리 시 브라우저 메모리 사용량도 수십 MB 수준으로 낮았습니다.

ECG-GRAFH Ruler for Tablet 메디컬에이아이

2025.02 ~ 2025.05

태블릿에서 확대·이동·정밀 측정이 서로 방해하지 않도록 터치 입력을 판별하고 제어한 심전도 그래프 측정 도구

React TypeScript D3 SVG Playwright

핵심 기여

한 가지 터치 입력에서 여러 동작을 구분

- ❗ PC에서는 마우스 버튼과 키보드로 확대, 이동, 측정을 구분할 수 있었지만 태블릿에서는 모두 손가락 터치로 시작했습니다. 실제로 핀치 확대 중 측정점이 생기거나 그래프를 이동하려다 측정이 시작되는 문제가 있었습니다.
- ✅ 터치가 시작된 순간 바로 기능을 정하지 않고, 유지 시간과 이동 거리, 손가락 수를 보고 사용자의 의도를 판단하도록 했습니다. 한 동작이 시작된 뒤에는 다른 기능이 같은 입력을 동시에 처리하지 않도록 공통 제어 상태를 두었습니다.
- 🔍 확대, 이동, 측정, 정밀측정, 도구 드래그가 같은 화면 위에서 동작하더라도 서로 입력을 빼앗는 문제를 줄였습니다. 화면 좌표가 실제 시간과 전압 값으로 변환되기 때문에 좌표 변환과 상태 로직은 단위 테스트로, 입력에 따른 상태 전이와 측정점 생성·제거는 모바일 디바이스 에뮬레이션을 포함한 브라우저 통합 테스트로 검증했습니다.

기술 스택

Frontend	React	TypeScript	Next.js	Zustand	TanStack Query	React Hook Form	Zod	D3
	styled-components	Base UI						
Testing	Vitest	Playwright	Testing Library	Mock Service Worker				
Tooling	Vite	Storybook	Yarn	pnpm	GitHub Actions	GitLab CI/CD		

자격 및 언어

자격증	SQLD
언어	English (OPIc IM2)